Project Build Process

This guide describes how to move from an early software idea to a working, tested product slice. It is meant to help the project owner make engineering decisions and learn the reasoning behind them, not just produce code quickly.

The process has stages, but it is not a one-way waterfall. New evidence or a failed assumption can send us back to an earlier stage. Keep the work small: resolve what the next slice needs, and leave later questions visible without trying to answer them all now.

1. Capture the idea as a hypothesis

Describe the possible user, their situation, the difficulty they face, and the change the software might make.

For example:

> Short-let operators may have trouble knowing whether a unit has been inspected after checkout. HomeRedi could help caretakers record that inspection.

This is a hypothesis, not yet a confirmed problem or a product requirement. Keep assumptions separate from facts.

Output: a short problem hypothesis, including who may have the problem and what evidence is missing.

2. Research the problem

Look for evidence about the user’s real work before committing to a solution. Useful methods include:

- observing the work as it happens;
- asking people to describe recent, specific examples;
- reviewing the tools, records, and workarounds they use now;
- checking reliable existing research and product documentation.

Ask about what people did, how often it happens, what goes wrong, and what the effects are. Do not treat a person's general interest in an idea as proof that they will use or pay for it.

Record the source and date of evidence. Label conclusions as observed facts, user reports, or your own assumptions. Research should be able to change the idea, not only support it.

Output: an evidence summary, the current understanding of the problem, and remaining research questions.

3. Decide whether the problem is worth pursuing

Use the evidence to decide whether the problem is frequent or costly enough for the intended users to care about. Compare it with their current way of working and other problems they have.

If the evidence is weak, do more research, change the target user or problem, or stop pursuing the idea. Do not turn an untested assumption into a product claim.

Output: a reasoned decision to continue, revise, investigate further, or stop.

4. Choose one small product slice

Describe one useful workflow from its trigger to its outcome. Include:

- who is involved;
- what event starts the workflow;
- what the person does;
- what the system does;
- what result ends the workflow;
- what is explicitly outside this slice.

Keep it small enough to specify, build, and check as one coherent behavior. A slice should deliver a result to someone; it should not be a list of technical tasks such as "create the database tables."

CONFIDENTIAL

Output: a short workflow description and an in-scope/out-of-scope list.

5. Find and resolve the decisions that affect the slice

This stage is not “think of every possible question.” Start with the workflow and look for places where the description is vague or where two reasonable interpretations would make the software behave differently.

5.1 WRITE THE WORKFLOW AS A SEQUENCE

Use a plain sentence for each step:

> An event happens. Someone or another system sends information. HomeRedi identifies the relevant record, checks rules, changes data, and shows or sends a result.

For each step, ask:

- **People and systems:** Who starts this? Who acts next? What does each person or system need?
- **Terms:** What does each important word mean in this workflow?
- **Inputs:** What information is needed, and where does it come from?
- **Rules:** What must be true? What determines the result?
- **State and records:** What is created, changed, or kept for later?
- **Timing and repetition:** What if the event is repeated, late, or out of order?
- **Failures:** What can go wrong, and what should the system do then?
- **Outcome:** How can the person or calling system tell what happened?

These prompts help reveal uncertainty. They are not a requirement to design every edge case before any work can begin.

5.2 TURN UNCERTAINTY INTO A DECISION RECORD

For each uncertainty that could change behavior, data, or what a person can do, record:

CONFIDENTIAL

Field	What to write
Question	The specific behavior or rule that is unclear
Options	Realistic answers, including “do more research” or “remove this from the slice”
Evidence	What supports or weakens each option; label assumptions clearly
Consequences	What changes for users, data, operations, or future work
Decision	The choice for this slice, or “open” if there is not enough basis to choose
Reason	Why the choice is appropriate given the evidence and scope

Do not choose an answer only to keep moving. If the answer depends on facts about users or another system, research it. If the question is not needed for this slice, explicitly defer it. If it blocks a testable behavior and cannot yet be answered, narrow or pause the slice.

5.3 USE A DECISION GATE

Before writing the specification, check:

- Can we explain the expected outcome for the main path?
- Are the terms and inputs clear enough to avoid competing interpretations?
- Have we decided what happens for the important failure or repeat cases in scope?
- Are the remaining open questions either outside this slice or clearly bounded?

The gate is about the current slice only. Do not block it on decisions that matter only to future features.

Output: a short list of decisions, evidence, and deferred questions.

6. Write the specification

Turn the decisions into examples that a person can check. A useful format is:

> Given **a known starting state**, when **an actor or system performs an action**, then **the observable result is**.

CONFIDENTIAL

Write examples for the main path and important in-scope failure or repeat cases. Include the state and data changes that matter. Avoid describing implementation details as requirements unless they affect behavior or constraints.

Diagrams can show workflow steps, state changes, or data relationships. They support the specification, but they do not settle missing rules by themselves. A draft diagram should be labelled as draft, and its assumptions should be recorded.

Review the examples against the decision record. Every behavior in the examples should have a basis, and every in-scope decision should be reflected in the examples.

Output: acceptance criteria or example scenarios, with diagrams that agree with them.

7. Plan the code shape

Now decide where the specified behavior belongs in the existing project. Start from the behavior and trace one thin path through the application:

> request or event → application behavior → domain rule → persistence or other side effect → response or result

Choose the smallest set of code parts that can implement that path. For a Spring Boot feature, this might include a controller or event handler, an application service, domain objects, repository code, and request/response types. Add only the parts the behavior needs.

Organize code by business capability or feature, with technical layers inside that capability. Do not create a package for every entity by default. Keep ownership clear: one capability owns a record’s lifecycle; other capabilities use it through a clear boundary rather than maintaining competing authoritative copies.

Decide which test level fits each rule:

- use a unit test for an isolated business rule;
- use an integration test when database behavior or Spring wiring is part of the behavior;
- use an API-level test when the request and response contract is important.

CONFIDENTIAL

Do not design the whole future architecture for a small slice. Record larger architecture questions and revisit them when the project has evidence that they matter.

Output: a small implementation sequence, package placement, and a test approach tied to the acceptance criteria.

8. Implement with TDD

For each behavior:

1. State the behavior and important cases in plain language.
2. Write the smallest test that demonstrates one case.
3. Run it and confirm it fails for the expected reason.
4. Write the smallest implementation that makes it pass.
5. Run the relevant tests again.
6. Refactor while keeping the tests passing.

Work in small steps. Explain unfamiliar syntax as it appears. Use compiler messages, test failures, IDE help, and official documentation as normal tools; memorizing all syntax is not the goal.

If the test exposes a missing product rule, stop and resolve that rule. Do not make up behavior inside the implementation just to get a green test.

Output: working code whose important behavior is protected by relevant tests.

9. Check the whole slice

Verify the complete path that the user or another system will use. Check relevant tests, data changes, responses, and important failure behavior. Review whether the implementation still matches the specification and project design decisions.

Update the documentation when implementation reveals a real change or correction. Keep the reason for that change visible; do not let the code and specification silently drift apart.

Output: a checked slice, a clear record of what works, and known limitations.

CONFIDENTIAL

10. Learn from use and choose the next slice

Put the software in front of intended users or use it in a realistic setting. Observe what happens, collect specific feedback, and compare the result with the original problem hypothesis.

Use what you learn to revise the product, do more research, or choose the next slice. A completed implementation is not proof that the original problem was important or that the solution is valuable.

Output: updated evidence, product decisions, and a justified next step.

Worked example: checkout to inspection creation

The current HomeRedi scope describes this flow:

> An external checkout trigger arrives. HomeRedi creates a pending inspection and pending tasks for the unit's spaces, changes the unit inspection status, and makes the inspection available for the next step.

Walking through that story exposes questions such as:

- What identifies the unit in the trigger?
- Where are the unit's spaces stored, and what counts as one space?
- What identifies the checkout so a repeated trigger can be recognized?
- Who should be able to act on the new inspection?
- Which records and statuses change together if part of creation fails?

These are not automatically all requirements. For each one, use the decision record: inspect the existing workflow and evidence, list realistic options, compare consequences, then decide, research, or defer. For example, if caretaker assignment is not part of the approved slice, record it as deferred rather than inventing an assignment rule.

Once the needed decisions are made, write acceptance examples. One possible example shape is:

CONFIDENTIAL

> Given a unit with its configured spaces, when a valid checkout trigger is accepted, then one pending inspection is created with one pending task per configured space, and the unit's inspection status becomes `AWAITING_INSPECTION`.

That sentence is only ready to become a test after "configured spaces," "valid trigger," and the behavior for repeated triggers have been decided for the slice. Then plan the smallest code path and write the first test for the first specified behavior.

For HomeRedi's current scope and approved decisions, see PROJECT_STATUS.md. If this guide and that file differ on current product scope, follow `PROJECT_STATUS.md` and update this guide when needed.

CONFIDENTIAL